

CS4800: Designing Sustainable ICT Systems

Lecture 4: Sustainable AI – Part 1

September 18, 2026

Lecturer: Przemysław Pawełczak

Authors: Liwia Padowska, Przemysław Pawełczak

CS4800-EWI@tudelft.nl

What will you learn in this lecture?

01

Instruction-Level Parallelism

Explain instruction-level parallelism and why executing multiple instructions at once has practical limits.

02

The Power and Memory Walls

Describe the power and memory walls and explain why traditional CPU performance scaling has slowed.

03

The Energy-Efficiency Ceiling

Understand the energy-efficiency ceiling imposed by the physical limits of modern chip technology.

Why hardware efficiency is a sustainability problem

1.5%

of all global electricity already goes to data centers, growing about 4x faster than total electricity demand.

IEA, Energy and AI, 2025

~30%

of an AI data center's carbon footprint comes from making the chips, before they ever run a single job.

Semiconductor Digest / imec, 2025

~75%

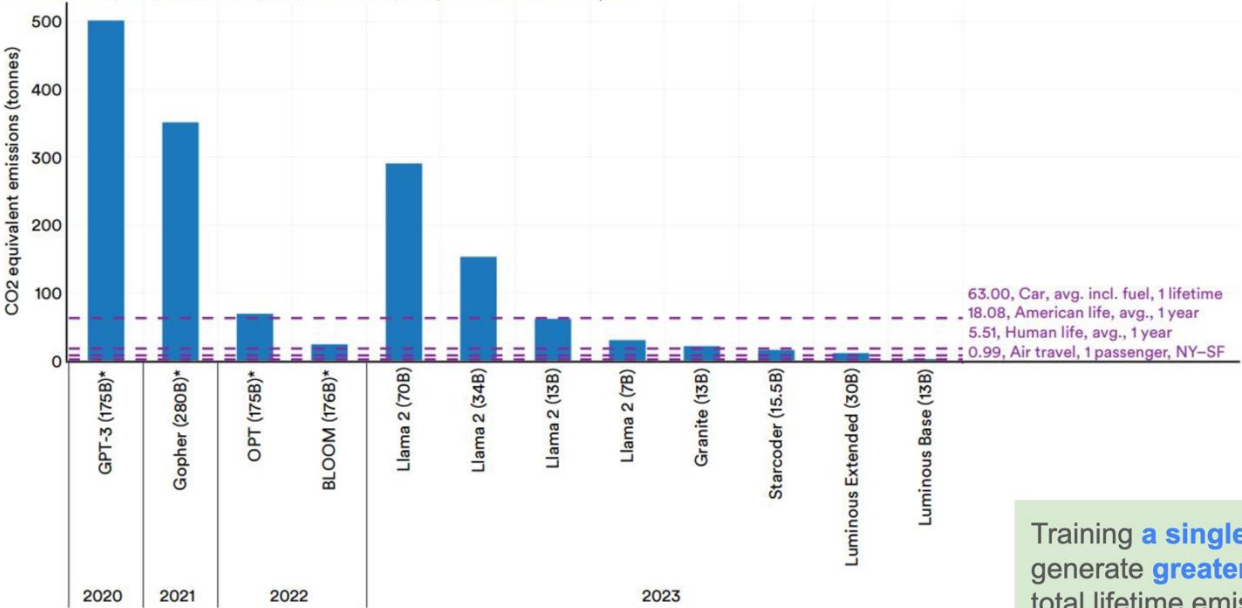
of a laptop's lifetime carbon footprint is already spent by the time you turn it on for the first time.

Dell & TCO Certified life-cycle studies

Why hardware efficiency is a sustainability problem

CO2 equivalent emissions (tonnes) by select machine learning models and real-life examples, 2020–23

Source: AI Index, 2024; Luccioni et al., 2022; Strubell et al., 2019 | Chart: 2024 AI Index report



Training a single large language model can generate greater CO2 emissions than the total lifetime emissions of 8 cars (in 2020)!

Environmental impact of select models

Source: AI Index, 2024; Luccioni et al., 2022 | Table: 2024 AI Index

Model and number of parameters	Year	Power consumption (MWh)
Gopher (280B)	2021	1,066
BLOOM (176B)	2022	433
GPT-3 (175B)	2020	1,287
OPT (175B)	2022	324
Llama 2 (70B)	2023	400
Llama 2 (34B)	2023	350
Llama 2 (13B)	2023	400
Llama 2 (7B)	2023	400
Granite (13B)	2023	153
Starcoder (15.5B)	2023	89.67
Luminous Base (13B)	2023	33
Luminous Extended (30B)	2023	93

Training a single model can consume more than 1000 MWh of power!

PART 1 - PHYSICAL LIMITS OF COMPUTING

Why CPUs Stopped Getting Faster (and What Comes Next)

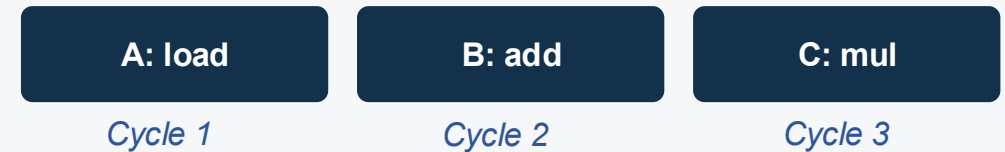
1.1 What is Instruction-Level Parallelism?

A Central Processing Unit (CPU) core can run multiple instructions at once, if they **do not depend on each other**.

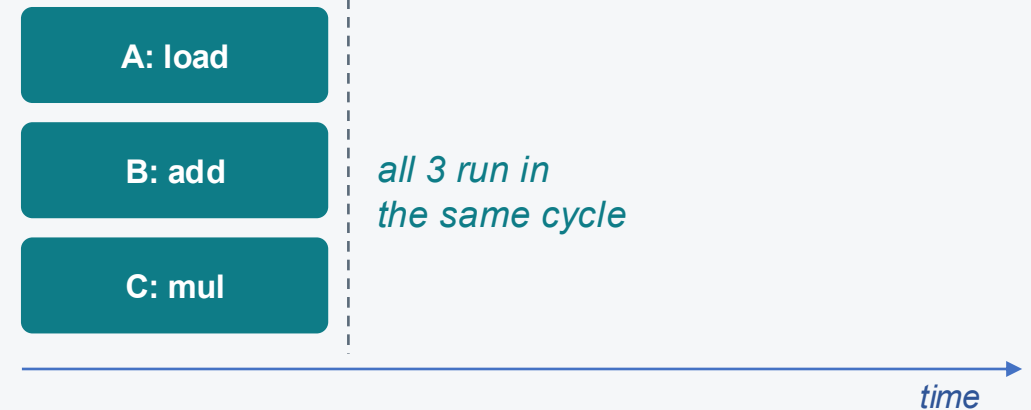
- The CPU scans upcoming instructions for ones that are independent
- Independent instructions are overlapped in time instead of run sequentially
- This happens automatically and is invisible to the programmer

Sequential vs. Overlapped Execution

Without Instruction Level Parallelism (ILP)



With ILP (independent instructions overlap)



1.2 Two issues preventing ILP

1. Dependencies

If instruction B needs the result of instruction A, B must wait.

```
ADD r1, r2, r3;    r1 = r2 + r3  
  
MUL r4, r1, r5;    needs r1!
```

Consequence:

MUL can't start until ADD finishes writing r1.

This is a **Read-After-Write** dependency.

2. Uncertainty

Branches (if/else) and memory accesses create outcomes the CPU can't know in advance.

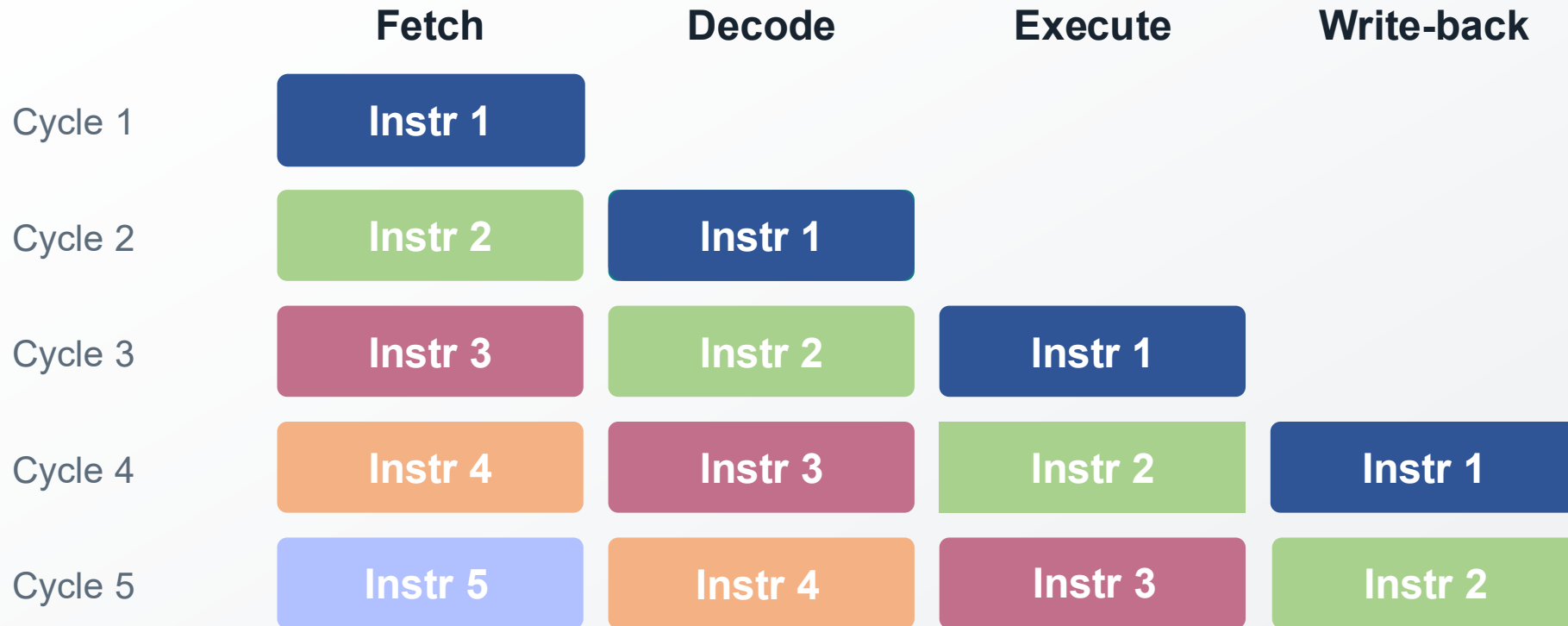
```
if (x > 0)  
{ y = a+b; }  
else  
{ y = a-b; }  
z = y * 3;
```

Consequence:

The CPU predicts the branch before x is resolved. If **branch misprediction** occurs, the CPU wastes processing time and energy executing the wrong instructions.

1.3 CPU Pipelining: an assembly line for instructions

- Every instruction passes through 4 stages: Fetch → Decode → Execute → Write-back.
- Pipelining allows multiple instructions to use different stages simultaneously.



- All 4 execution stages stay busy every cycle.
- This raises throughput even though each instruction still takes 4 cycles.

1.4 Superscalar issue and out-of-order execution

Superscalar: do several at once

If instructions are independent AND a free execution unit of the right type exists, the core runs several instructions per cycle.

```
x = a + b;    y = c * d;    z = e - f;
```

ALU 1: $a+b \rightarrow x$

ALU 2: $c*d \rightarrow y$

ALU 3: $e-f \rightarrow z$

All 3 instructions complete in 1 cycle instead of 3.

But only if 3 ALUs are free!

Out-of-order: don't wait idly

While a slow instruction is pending, the core runs later independent instructions early.

Then it commits results in the original order.

Instr 1: load x (slow, ~100 cycles)

Instr 3: $z=a+b$ ✓

Instr 4: $w=c*d$ ✓

Instr 2: $y=x+1$ (runs once x arrives)

A reorder buffer tracks the original order, so results are still committed chronologically.

1.4 Speculative execution: guess, then verify

```
if (x > 0)
    { y = a+b; }
else
    { y = a-b; }
z = y * 3;
```

The CPU predicts the branch and starts executing immediately, **without** waiting for the condition to resolve.

No speculation

1: fetch x from RAM
...**waiting (100 cycles)**...
100: x arrives
101: run y=a+b
102: run z=y*3

Total: ~102 cycles

Correct guess

1: fetch x
2: **GUESS** x>0 → run y=a+b
3: run z=y*3
100: x confirmed TRUE
101: commit results

Total: ~101 cycles,
but **z** **computed** in cycle 3

Wrong guess

1: fetch x
2: **GUESS** x<0 → run y=a-b
3: run z=y*3
100: x is -5 (FALSE!)
101: discard y,z
102-103: redo else-branch

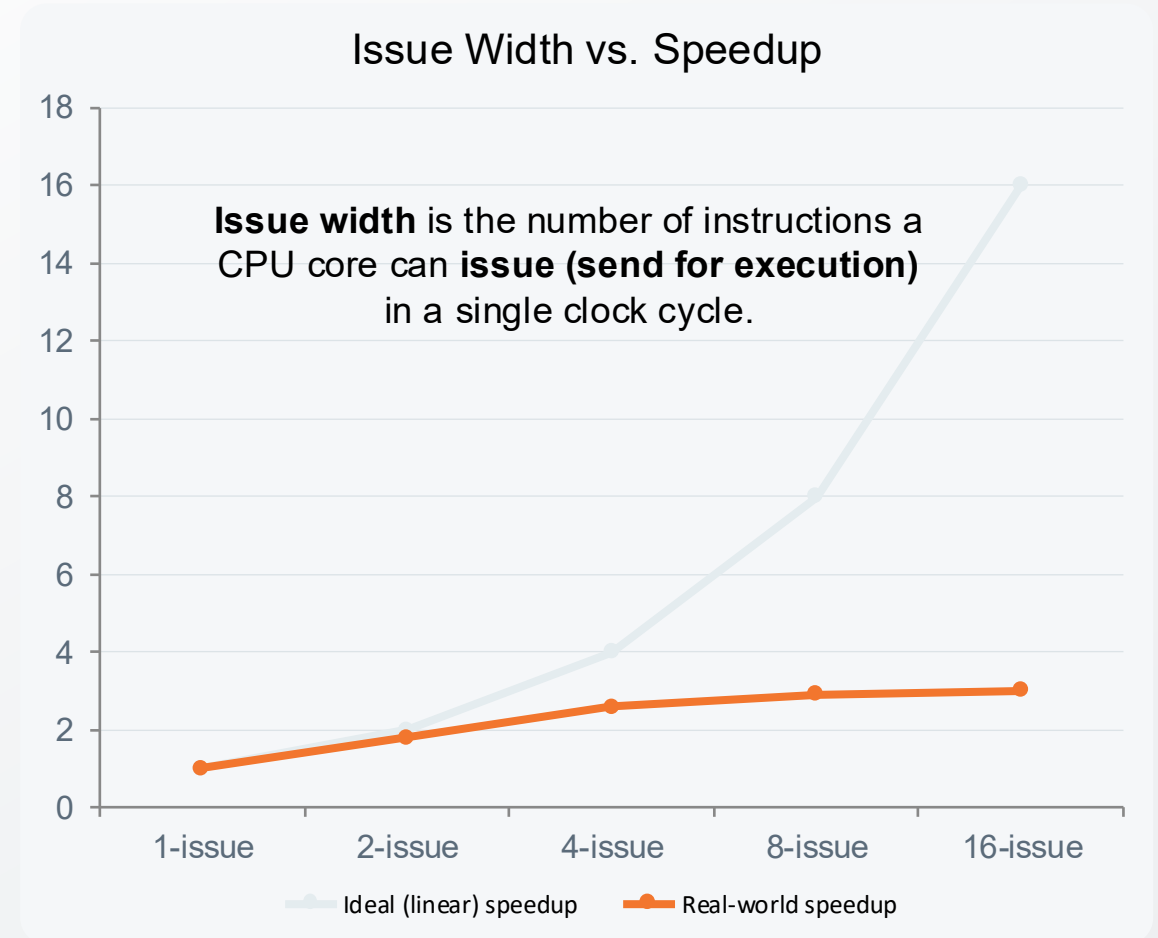
Total: ~103 cycles,
y and **z** need to be **recomputed**

1.5 The ILP wall: wider cores, diminishing returns

More issue width does not always mean more speed:

- [Wall \(1991\)](#): Real programs typically expose only 5 to 7 parallelizable instructions, even under ideal conditions.
- A 4 issue CPU may not have 4 independent instructions ready each cycle.
- An 8 issue CPU gains little if the program only provides 2 to 3 independent instructions.

[Wall, D. W. \(1991\). *Limits of Instruction Level Parallelism*. ACM SIGPLAN Notices. 26\(4\). 176–188.](#)



Source: [Elmashar, A., & Aljahadli, S. \(2013\). Experimental and theoretical speedup prediction of MPI-based applications.](#)

PART 1 — THE WALLS

The ILP Wall is the first limit to speedup.

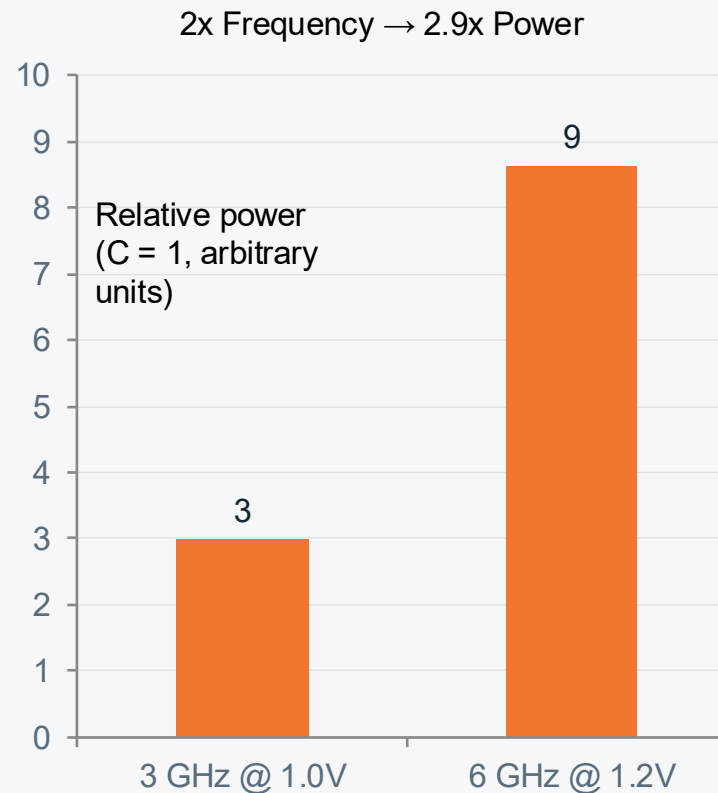
What about power?

1.6 The Power Wall: why GHz stopped increasing

Complementary Metal Oxide Semiconductor (CMOS) is the technology used to build most modern CPUs. It uses transistors as switches, which consume power when they change state. Higher frequency increases power, while higher voltage increases it even faster.

$$P \approx C \cdot V^2 \cdot f$$

- C: switched capacitance
- V: supply voltage
- f: clock frequency
- Doubling frequency roughly doubles power
- Voltage is squared (bigger impact)



Why this kills the “free lunch”

- More CPU power means more heat, requiring more cooling.
- Cooling systems have physical, space, noise, and cost limits.
- In a data centre, doubling performance may require much more than double the power, making it increasingly expensive and difficult to cool.
- [Sutter \(2005\), *The Free Lunch Is Over*](#)

1.7 End of Dennard scaling → Dark Silicon

Dennard scaling (until ~2005)

Shrink a transistor → scale down voltage & current
→ power density stays roughly constant while speed improves

Before ~2005:

smaller transistor → lower voltage → higher frequency

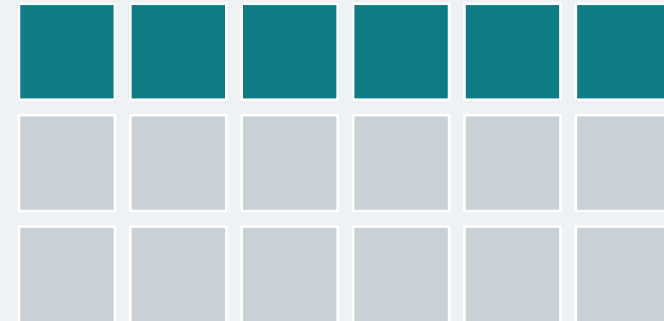
▼ ~2005 ▼

After ~2005:

voltage can't drop much further, so leakage current becomes a much bigger share of total power

Dark silicon

Transistor budgets kept growing, but the chip's power budget didn't grow to match → not everything on the chip can be switched on at once:



 *Lit = powered core*  *Dark = unpowered silicon*

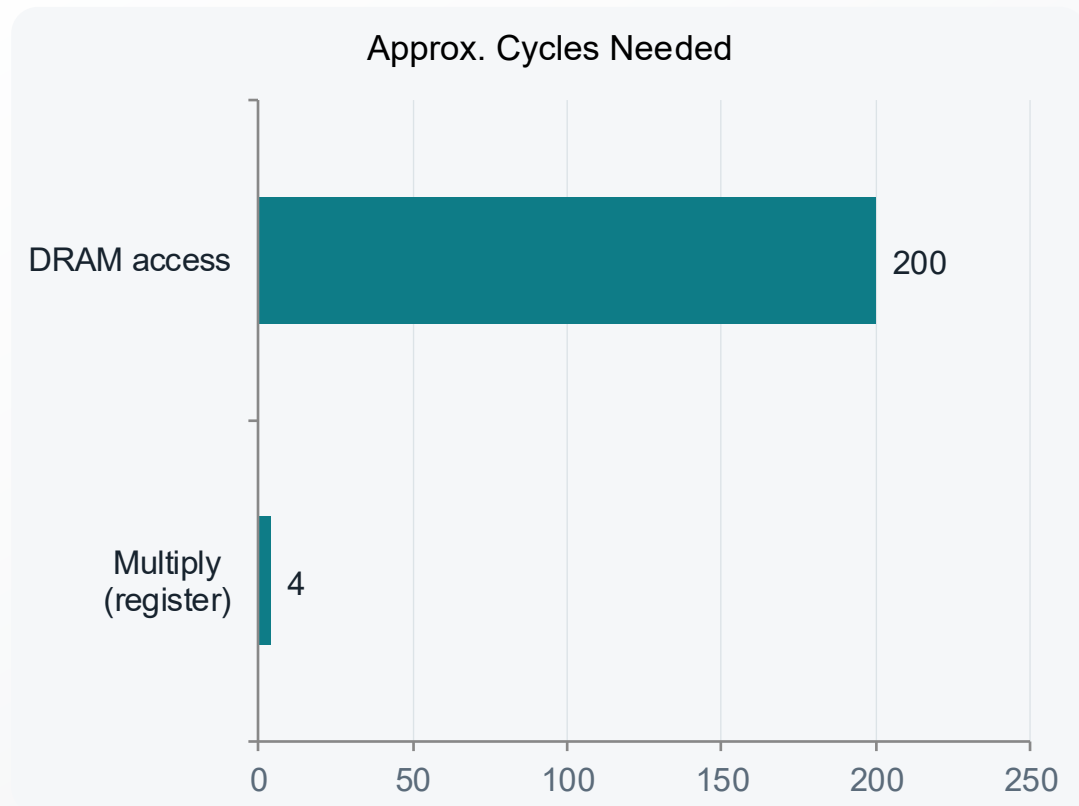
Result: designers pivoted to more cores, bigger caches, and specialised accelerators instead

PART 1 — THE WALLS

Does memory limit us too?

1.8 The Memory Wall: compute got fast, memory didn't

Modern CPUs can do dozens of arithmetic operations in the time it takes to fetch one value from main memory (DRAM).



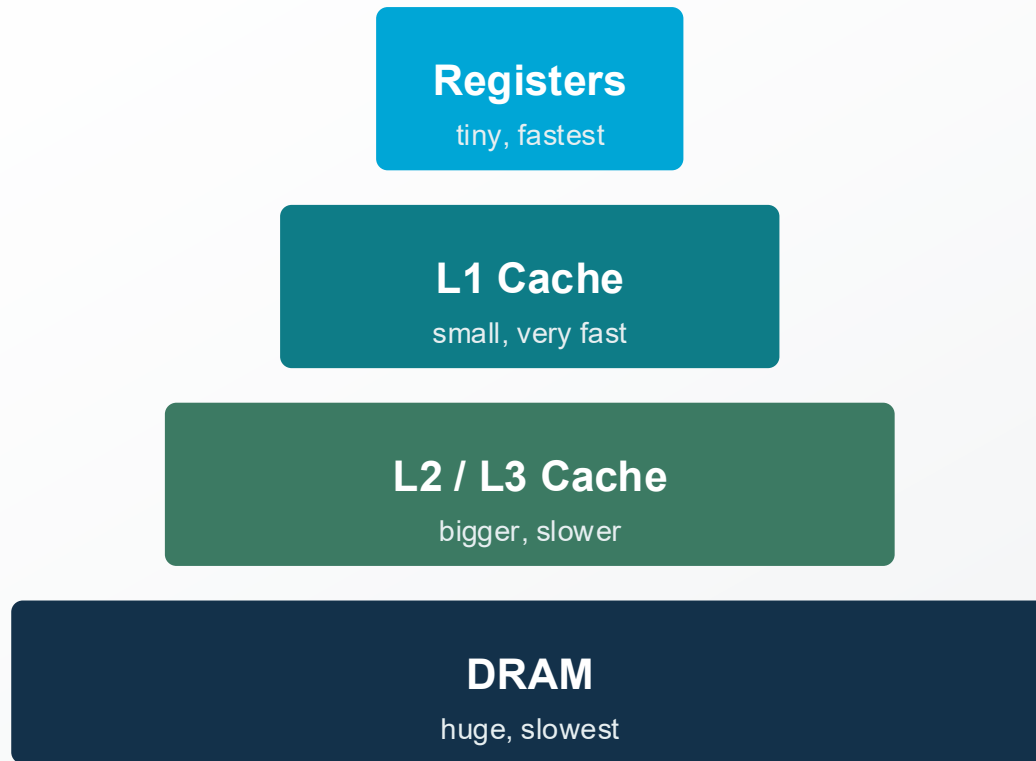
Latency vs. Bandwidth

Latency: how long it takes to receive the first byte of data. High latency causes stalls, because the next step can't start until it arrives.

Bandwidth: how much data can be transferred per second once the transfer has started.

Why graphs vs. ML training differ: graphs are latency-bound: visiting the next node means first resolving a pointer from the current one, so each access waits on the last. ML training is bandwidth-bound: it streams large, independent batches of weights and activations, so throughput (not per-access delay) sets the pace

1.9 Caches: hiding memory latency with locality



Average Memory Access Time (AMAT)

$$AMAT = T_{hit} + (Miss_{rate} \times Miss_{penalty})$$

- T_{hit} : time when the data IS in the cache (a few cycles)
- $Miss_{rate}$: fraction of accesses NOT found in cache
- $Miss_{penalty}$: extra time to fetch from L2/L3/DRAM after a miss (tens to hundreds of cycles)

Because the miss penalty is multiplied in, even a small miss rate can significantly raise AMAT. This is why locality (temporal & spatial reuse) matters so much.

1.10 Fighting the memory wall: locality, prefetching, OoO

Locality (matrix example)

```
// GOOD: row-by-row
for i: for j:
    sum += A[i][j];

// BAD: column-by-column
for j: for i:
    sum += A[i][j];
```

Same computation, same hardware.

The bad version can be 5-10x slower purely from cache misses.

Prefetching

The hardware watches access patterns and loads data into cache before it's requested.

```
A[0], A[1], A[2] ...
-> prefetch A[3], A[4]
```

Works great for sequential access.

Fails for pointer-chasing (linked lists). The next address is unknown until the previous load finishes.

Out-of-order fill-in (OoO)

While one load stalls, the core executes other independent instructions to keep busy.

```
x = load(mem[100]);
//slow
y = x + 1;    // blocked
z = a + b;    // runs now
w = c * d;    // runs now
```

This only covers the gap if enough independent work exists. In memory-bound code, it often doesn't.

PART 1 — THE WALLS

One physical ceiling behind the three walls

1.11 The physical ceiling on FLOP per Joule (1/2): E_transistor

Even a perfectly-engineered CMOS chip can't get infinitely energy-efficient. The first of two costs that set this ceiling: the energy to switch a transistor.

E_transistor

$$E_{\min} = k_B \cdot T \cdot \ln 2 \approx 2.9 \times 10^{-21} \text{ J (at } T = 300 \text{ K)}$$

Energy to switch transistors. Floor set by Landauer's principle ($k_B T \ln 2$ per bit erasure).

But real reliability margins push practical energy 1-2 orders of magnitude above that.

1–2 orders

of magnitude above the Landauer floor is roughly where today's real transistors operate, once reliability margins and non-ideal switching are included.

1.11 The physical ceiling on FLOP per Joule (2/2): E_interconnect

The second cost that caps energy per FLOP: moving signals across the wires that connect transistors together.

E_interconnect

$$E \propto C_L \cdot L \cdot N \cdot V^2$$

Energy to charge/discharge wires: $E \propto C_L \cdot L \cdot N \cdot V^2$.

Even if logic gets nearly free, moving signals across wires still costs. And this cost hasn't improved much historically.



Scope: this bounds CMOS operating irreversibly under today's paradigm.
It does not bound future approaches like optical/spin-based computing.

$$\sim 4.7 \times 10^{15}$$

FP4 operations per Joule
(estimated ceiling, Ho et al. 2023)

*$\approx 200\times$ headroom vs. today's
best dense GPUs*

Exercise

Task: Out-of-order execution

```
int f(long *m, int a, int b,
      int c, int d) {
    int x = m[100]; // slow
    Load
    int y = x + 1; // waits
    on x

    int z = a + b; //
    independent
    int w = c * d; //
    independent
    return y + z + w;
}
```

Before you check: look at the function above and predict how many of its lines can execute in the same CPU cycle, and why.

Then verify it on godbolt.org:

1. Open godbolt.org in your browser.
2. Paste the function above into the left-hand source pane.
3. Set the compiler to a recent clang and add the flag -O2.
4. Click “Add tool” and select llvm-mca (Machine Code Analyzer).
5. Open the Timeline view and find the rows for `y = x + 1`, `z = a + b`, and `w = c * d`.
6. Compare what you see to your prediction: were **z** and **w** really independent of the slow load, and did the CPU run them early?

Exam(ple) questions

1. Which statement about reading energy through the `MSR_PKG_ENERGY_STATUS` register is false?
 - a. The register is read with the x86 RDMSR instruction, which requires root access
 - b. The raw counter is 32 bits wide and wraps back to zero roughly every 60 seconds under heavy load
 - c. The scale factor that converts raw ticks into joules is read from a separate register, `MSR_RAPL_POWER_UNIT`
 - d. Reading the counter once, right when your code finishes running, tells you exactly how many joules that run used
2. Which statement about the batching example (buffering log events instead of sending one HTTP call per event) is false?
 - a. The batched code sends the whole buffer in a single HTTP call once `BATCH_SIZE` is reached or a timer elapses, whichever comes first
 - b. In the unbatched version, every single event wakes the radio out of its low power state, even if several events arrive close together
 - c. The savings come from letting the radio return to a low power idle state between transmissions instead of waking it for every event
 - d. Batching removes the network calls entirely and replaces them with local storage, so no data reaches the server until the next deploy

Exam(ple) questions

1. Which statement about reading energy through the `MSR_PKG_ENERGY_STATUS` register is false?

- ~~a. The register is read with the x86 RDMSR instruction, which requires root access~~
- ~~b. The raw counter is 32 bits wide and wraps back to zero roughly every 60 seconds under heavy load~~
- ~~c. The scale factor that converts raw ticks into joules is read from a separate register, `MSR_RAPL_POWER_UNIT`~~
- ~~d. Reading the counter once, right when your code finishes running, tells you exactly how many joules that run used~~

2. Which statement about the batching example (buffering log events instead of sending one HTTP call per event) is false?

- ~~a. The batched code sends the whole buffer in a single HTTP call once `BATCH_SIZE` is reached or a timer elapses, whichever comes first~~
- ~~b. In the unbatched version, every single event wakes the radio out of its low power state, even if several events arrive close together~~
- ~~c. The savings come from letting the radio return to a low power idle state between transmissions instead of waking it for every event~~
- ~~d. Batching removes the network calls entirely and replaces them with local storage, so no data reaches the server until the next deploy~~

Recap

ILP wall

Real code exposes only ~5-7 independent instructions so wider cores stop paying off

Memory wall

DRAM access (~200 cycles) is ~50× slower than a multiply (~4 cycles).
Caches reduce this cost, but cannot eliminate it.

Power wall

$P \approx CV^2f$ shows that increasing clock frequency raises power consumption and heat, while voltage has an even stronger (quadratic) effect, making further GHz scaling increasingly difficult and expensive to cool

Energy-efficiency ceiling

CMOS cannot achieve unlimited FLOP/J: transistor switching has a physical energy floor, and moving data across interconnects also consumes energy

Lecture Feedback

